

【Learn Claude Code 源码详解】- 让 Hello 进入循环

开发实战分享屋

2026/04/13

目录

1	环境准备与断点切入	2
2	外层循环与用户输入	3
2.1	外层 <code>while True</code> 循环的作用	3
2.2	等待用户输入: <code>query = input(...)</code> (第 94 行)	3
2.3	将输入封装为结构化消息: <code>history.append(...)</code> (第 99 行)	4
2.4	本章小结与下章衔接	4
3	Agent Loop 的发送与接收	5
3.1	进入 <code>agent_loop</code> : 内层的智能体循环	5
3.2	通过 <code>Client</code> 发送 LLM 请求	5
3.3	追加响应到消息列表	7
3.4	判断停止原因: <code>end_turn</code> 与 <code>tool_use</code>	7
3.5	本章小结与衔接	8
4	响应解析——ThinkingBlock 与 TextBlock	8
4.1	从 <code>history[-1].content</code> 提取响应体	8
4.2	响应体的双块结构: <code>ThinkingBlock</code> 与 <code>TextBlock</code>	9
4.3	第一个块: <code>ThinkingBlock</code> (思考块)	9
4.4	第二个块: <code>TextBlock</code> (文本块)	9
4.5	本章小结	10
5	流程总结与下节预告	10
5.1	无 <code>Tool Use</code> 场景的完整交互流程	10
5.2	下节预告: <code>Tool Use</code> 分支	11

1 环境准备与断点切入

本门课程承接上一节的内容，UP 主不再重复网页端的动画演示，而是直接进入 VS Code 的源码 Debug 环节，通过逐行跟踪的方式，带领大家理解 learn-claude-code 项目中 Agent Loop 的启动过程与初始状态¹。

在正式开始 Debug 之前，UP 主简要提到了网页端的动画演示。该动画非常详细地展示了用户与大模型之间的交互过程，如果你在本地已经把项目启动起来，可以花更多时间自行观看²。不过，为了更高效地深入源码，UP 主选择跳过动画，直接回到 VS Code 中进行代码调试³。

Debug 的第一步是在主程序 `main.py` 的第 91 行设置断点。该行代码初始化了 `history` 变量，它是一个空列表，用于存储整个对话过程中的消息历史。随后在左侧点击 **Run and Debug** 启动调试器⁴。片刻之后，程序成功启动，调试器在第 91 行暂停，此时我们可以在 VS Code 的变量面板中清楚地看到 `history` 的类型为 `list`，且当前为空列表，如图 1 所示。

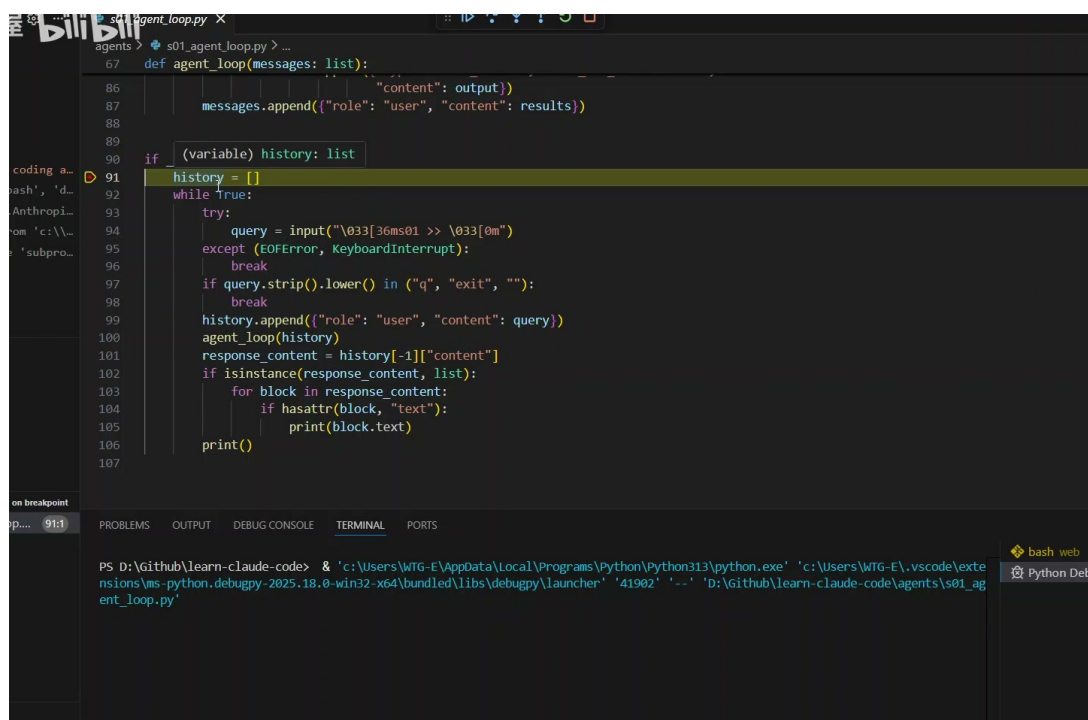


图 1: 在 `main.py` 第 91 行设置断点并启动 Debug，`history` 初始化为空列表

从代码结构上看，第 91 行的 `history = []` 是整个外层交互循环的起点。`history` 作为 History（历史消息列表），其类型为 `list[dict]`，后续将不断追加用户输入和模型响应，作为上下文传递给大模型。这个变量的初始状态虽然简单，却是理解整个消息流转机制的关键：所有对话状态都集中保存在这一个列表中，没有额外的全局状态或外部存储⁵。

断点命中后，程序已经加载完成，接下来只需单步执行，即可进入第 92 行的外层 `while`

¹视频时间戳：00:00

²视频时间戳：00:08

³视频时间戳：00:25

⁴视频时间戳：00:37

⁵视频时间戳：00:53

True 循环。该循环保证了 Agent 可以持续与用户交互——只要不主动输入退出指令，程序就会一直等待新的 Query（用户查询），并将其送入后续的 Agent Loop 进行处理。至此，本章节的 Debug 准备工作已经全部完成，程序处于待命状态，等待用户输入第一个 Query。

从与下一章的衔接来看，断点命中、程序启动之后，下一步就是进入外层循环并在第 94 行通过 `input()` 等待用户输入。UP 主将在下一章输入 "Hello"，展示这条 Query 如何被接收、封装为第一条消息字典，并最终追加到 `history` 列表中，为进入核心的 `agent_loop` 函数做好准备。

2 外层循环与用户输入

断点命中第 91 行后，程序已经完成了初始化工作。此时 VS Code 的调试面板中可以看到 `history` 变量被定义为一个空列表，这意味着当前还没有任何对话记录。接下来，程序将进入外层的主循环，开始接收用户的自然语言输入，这也是 Agent 能够持续交互的起点。

2.1 外层 while True 循环的作用

根据视频画面中的行号，第 92 行是一个 `while True:` 语句⁶。这个外层循环是整个程序能够“一直运行下去”的关键机制。只要用户不主动输入退出指令（例如 `q` 或 `exit`），Agent 就会持续等待新的用户查询，并反复进入后续的处理流程。换句话说，这个循环保证了智能体循环（Agent Loop）可以在一次对话结束后立刻准备迎接下一次对话，而无需重新启动程序。

UP 主在视频中强调，正是因为有了这个外层循环，我们才能与 Agent 不断进行交互——你可以一直向它提问，或者让它帮你完成各种任务⁷。这种设计模式在命令行交互式应用中非常常见，它把“接收输入—处理逻辑—输出结果”这三个步骤无限重复，直到满足退出条件为止。

2.2 等待用户输入：`query = input(...)`（第 94 行）

程序继续单步执行，来到第 94 行。这里调用了 Python 内置的 `input()` 函数，终端光标开始闪烁，等待用户键入内容⁸。在 Debug 过程中，UP 主在终端中输入了 `Hello` 并按下回车键⁹。此时，字符串 "Hello" 被赋值给变量 `query`。在 VS Code 的变量监视区可以清楚地看到，`query` 的值正是我们刚刚输入的 `Hello`¹⁰。

从源码结构来看，`query` 在这一步扮演的角色就是用户查询（Query）。它是用户与 Agent 沟通的桥梁——所有后续的模型推理、响应生成，都是基于这个字符串展开的。因此，理解 `input()` 这一行代码，就是理解“用户意图如何进入程序”的第一步。

⁶视频时间戳：01:02

⁷视频时间戳：01:07

⁸视频时间戳：01:18

⁹视频时间戳：01:40

¹⁰视频时间戳：01:48

```

67 def agent_loop(messages: list):
68     ...
69     ...
70     ...
71     ...
72     ...
73     ...
74     ...
75     ...
76     ...
77     ...
78     ...
79     ...
80     ...
81     ...
82     ...
83     ...
84     ...
85     ...
86     ...
87     ...
88     ...
89     ...
90 if __name__ == "__main__":
91     history = []
92     while True:
93         try:
94             query = input("\033[36ms01 >> \033[0m")
95         except (EOFError, KeyboardInterrupt):
96             break
97         if query.strip().lower() in ("q", "exit", ""):
98             break
99         history.append({"role": "user", "content": query})
100        agent_loop(history)
101        response_content = history[-1]["content"]
102        if isinstance(response_content, list):
103            for block in response_content:
104                if hasattr(block, "text"):
105                    print(block.text)
106        print()
107
Running
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Github\learn-claude-code> & 'c:\Users\WTG-E\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\WTG-E\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '41902' '--' 'D:\Github\learn-claude-code\agents\s01_agent_loop.py'
s01 >> hello

```

图 2: 终端等待用户输入 Hello 的画面

2.3 将输入封装为结构化消息: `history.append(...)` (第 99 行)

拿到 `query` 之后, 程序会先做一个简单的分支判断: 如果输入既不是退出关键字, 也不是空字符串, 就会继续向下执行。由于 "Hello" 显然不是退出指令, 程序顺利进入第 99 行¹¹。

第 99 行的代码逻辑如下所示:

```
1 history.append({"role": "user", "content": query})
```

Listing 1: 将用户输入追加到历史消息列表

这行代码做了两件非常重要的事: 第一, 它将用户的自然语言输入 `query` 封装成了一个字典; 第二, 它将这个字典追加到了 `history` 列表中。字典包含两个键值对: `"role": "user"` 表示这条消息的发送者是用户, `"content": query` 则存储了具体的消息内容, 也就是 "Hello"。

此时, `history` 从一个空列表变成了包含一条消息的消息列表。UP 主在视频中特别指出, 这是一个字典, 里面有两个键值对, `role` 是 `user`, `content` 就是我们输入的提示词¹²。这种结构化的表示方式是后续与大模型 API 交互的基础——大模型通常不会直接接收一个裸字符串, 而是需要接收一个带有角色标识的消息列表, 以便区分用户输入和模型回复。

2.4 本章小结与下章衔接

本章我们跟踪了从外层 `while True` 循环到用户输入、再到消息封装的完整过程。关键变量 `query` 和 `history` 在这一阶段完成了它们的第一次“亮相”: `query` 保存了用户的原始输入,

¹¹ 视频时间戳: 02:00

¹² 视频时间戳: 02:13

`history` 则将这个输入转化为标准格式的消息记录。

当第 99 行执行完毕后，`history` 已经准备好被传递给核心的 `agent_loop(messages)` 函数。下一章，我们将跟随程序进入第 100 行，看看 Agent 是如何把这条消息发送给大模型、又是如何接收并处理模型响应的。

3 Agent Loop 的发送与接收

当外层循环把用户输入的 "Hello" 封装成一条消息并追加到 `history` 列表之后，程序在第 100 行正式踏入了整个 Agent 最为核心的领地——`agent_loop(messages)` 方法。从这一刻起，程序不再只是简单地读取键盘输入，而是开始与大模型进行真正的网络交互。本章将紧随视频中的 Debug 过程，逐行剖析 `agent_loop` 内部的发送、接收与结束判断逻辑，帮助读者理解在不涉及工具调用的场景下，Agent Loop 是如何运转的。

3.1 进入 `agent_loop`：内层的智能体循环

视频画面中，UP 主在输入 "Hello" 并回车后，程序执行到第 100 行的 `agent_loop(history)` 调用处¹³。这一步标志着控制权从主程序移交到了 Agent 的核心函数。值得注意的是，`agent_loop` 内部同样维护了一个 `while True` 循环——这意味着 Agent 与大模型的交互并非一次性请求-响应，而是一个可以持续多轮的对话过程。在本节的简单示例中，由于用户只是打了个招呼，这个内层循环仅运行一次便会退出；但在更复杂的场景（例如需要多轮推理或工具调用）中，这个循环可能会迭代多次。

图 3 以流程图的形式展示了 `agent_loop` 的全貌。可以看到，进入循环后的第一件事就是向大模型后端发送请求，而这也是本节重点讲解的内容。

3.2 通过 Client 发送 LLM 请求

`agent_loop` 刚一进入循环体，便通过 `Client` 对象向后端发起了一次大模型请求¹⁴。这里的 `Client` 是代码中预先配置好的 API 客户端，负责封装网络通信细节。视频中 UP 主特别提到，当前项目配置的模型是 GLM（智谱的模型），因此这次请求实际上是发往智谱的服务器端点¹⁵。

请求体中最重要的参数便是 `messages`——也就是从外层传入的 `history`。此时 `messages` 列表中只有一条消息：

```
1  [  
2      {"role": "user", "content": "Hello"}  
3  ]
```

Listing 2: 当前 `messages` 列表的内容

¹³视频时间戳：02:22

¹⁴视频时间戳：02:45

¹⁵视频时间戳：02:59

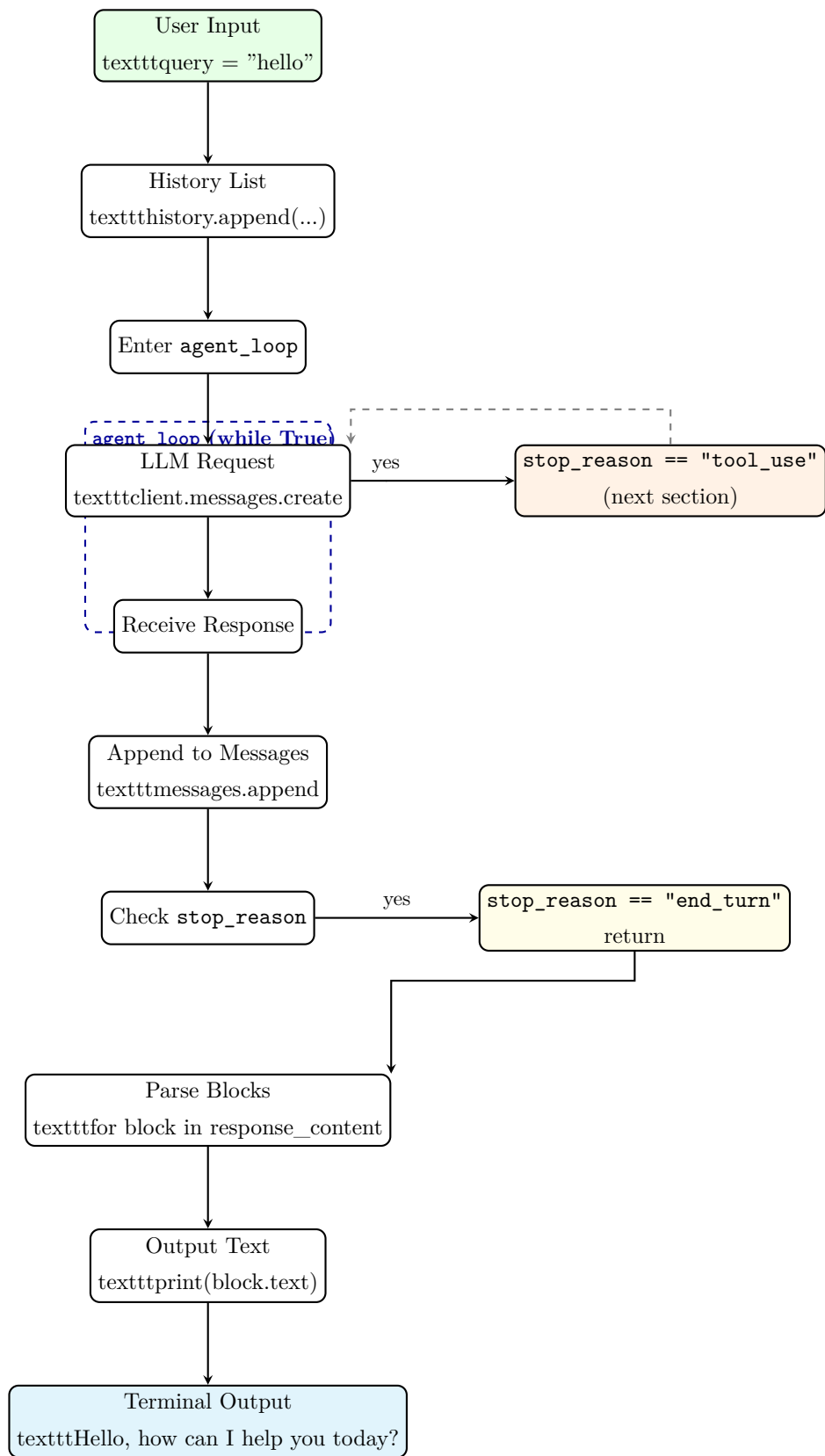


图 3: Agent Loop 的整体流程：从用户输入到模型响应，再到 `stop_reason` 判断与最终输出。

大模型收到这条消息后，会基于自身的预训练知识和上下文理解，生成一个完整的响应对象。这个响应对象在代码中被赋值给变量 `response`。`response` 并非单纯的字符串，而是一个结构化的对象，包含多个属性，其中两个最为关键的属性是：

- `response.stop_reason`：一个字符串，指示模型认为本轮交互应该以何种方式结束；
- `response.content`：一个列表，包含模型返回的具体内容块（Block），例如 `ThinkingBlock` 和 `TextBlock`。

关于 `response.content` 的详细解析，我们将在下一章深入展开；本章先把注意力集中在 `stop_reason` 的判断逻辑上。

3.3 追加响应到消息列表

在 `agent_loop` 内部，当 `response` 成功返回后，代码执行到第 74 行：`messages.append(response)`¹⁶。这一步看似简单，实则至关重要——它把大模型的响应正式纳入了对话历史，使得后续的循环迭代（如果有的话）能够基于完整的上下文继续推理。

需要强调的是，这里的 `messages` 与外层主程序中的 `history` 指向的是同一个列表对象。因此，当 `agent_loop` 内部执行 `messages.append(response)` 时，外层的 `history` 也会同步增长。视频 Debug 画面中可以看到，此时 `history` 列表中已经包含了两条消息：第一条是用户输入的 "Hello"，第二条就是大模型返回的 `response` 对象¹⁷。这种设计保证了消息历史在函数内外的一致性，也为后续在主程序中解析响应内容提供了便利。

3.4 判断停止原因：end_turn 与 tool_use

追加完响应后，`agent_loop` 来到了第 76 行，这里是决定循环命运的分叉口：

```
1 if response.stop_reason == "tool_use":
2     # 进入工具调用分支（下节讲解）
3     ...
4 else:
5     return
```

Listing 3: `agent_loop` 内部的 `stop_reason` 判断逻辑

视频中 UP 主将鼠标悬停在 `response` 对象上，展开其属性面板，清晰地显示出 `stop_reason` 的值为 "end_turn"¹⁸。这意味着 GLM 模型在分析完用户输入的 "Hello" 之后，判断本轮对话无需调用任何外部工具，可以直接给出文本回复并结束本轮交互。于是程序进入第 77 行的 `return` 语句，干净利落地跳出 `agent_loop` 的内层循环¹⁹。

`stop_reason` 是一个枚举性质的字符串属性，在本项目中主要取两个值：

¹⁶视频时间戳：03:12

¹⁷视频时间戳：04:11

¹⁸视频时间戳：03:42

¹⁹视频时间戳：04:05

- **"end_turn"**: 表示模型判断无需调用工具，直接给出文本回复，本轮对话正常结束（即停止原因中的“结束回合”）。
- **"tool_use"**: 表示模型判断需要调用外部工具以完成用户请求，Agent 必须进入工具调用分支执行相应操作（即“工具使用”场景）。

本节由于输入内容极为简单，`stop_reason` 自然落在了 **"end_turn"** 上。UP 主也在视频中明确指出，**"tool_use"** 分支才是 Agent Loop 中更为核心和复杂的部分，将在下一节课程中详细展开²⁰。

3.5 本章小结与衔接

综上所述，本章跟随 Debug 画面，完整呈现了 `agent_loop` 在一次无工具调用场景下的执行路径：

1. 第 100 行调用 `agent_loop(history)`，进入内层 `while True` 循环；
2. 循环伊始，通过 `Client` 向 GLM 后端发送请求，获取 `response`；
3. 第 74 行将 `response` 追加到 `messages`（亦即 `history`）列表；
4. 第 76 行检查 `response.stop_reason`，发现其为 **"end_turn"**；
5. 第 77 行执行 `return`，跳出 `agent_loop`，控制权回到主程序。

此时，`history` 中已经保存了两条消息，而 `response` 对象内部的 `content` 列表正等待着被进一步拆解。下一章，我们将把目光投向 `response.content`，看看模型到底返回了什么样的内容块，以及程序是如何从 `ThinkingBlock` 和 `TextBlock` 中提取出最终展示给用户的回答文本的。

4 响应解析——*ThinkingBlock* 与 *TextBlock*

`agent_loop` 返回之后，程序的执行流回到外层主程序的第 101 行（推断）。此时，`history` 中已经包含了两条消息：第一条是用户在第 99 行封装好的 **"user"** 消息，内容为我们输入的 **"Hello"**；第二条则是刚刚在 `agent_loop` 内部追加的模型响应。UP 主在这里停下脚步，开始逐层拆解这条响应的内部结构，向我们展示模型到底“想”了什么，以及最终“说”了什么。²¹

4.1 从 `history[-1].content` 提取响应体

在第 101 行，代码通过 `history[-1].content` 取出最后一条消息——也就是模型响应——的 `content` 字段，并将其赋值给变量 `response_content`。在 Python 中，`-1` 表示列表的最后一个元素，因此 `history[-1]` 精准地指向了模型返回的 `response` 对象。²²

²⁰ 视频时间戳：06:39

²¹ 视频时间戳：03:20

²² 视频时间戳：04:22

当我们将鼠标悬停在 `response_content` 上，或者在左侧的调试面板中展开该变量时，可以清晰地看到它的类型是一个列表（`list`），而非常规的字符串。这意味着，大模型返回的 `content` 并不是一段简单的文本，而是一个由多个块（**Block**）组成的结构化列表。²³

4.2 响应体的双块结构：ThinkingBlock 与 TextBlock

`response_content` 列表中包含两个元素，分别是 `ThinkingBlock` 和 `TextBlock`。这两个类型共同构成了模型响应的完整内容体：前者承载模型的内部思考过程，后者承载最终呈现给用户的回答文本。

为了更直观地理解这一结构，我们可以将其想象为如下的层次关系：模型在收到用户输入后，首先生成一段内部的推理过程（即 `ThinkingBlock`），这段内容通常不会直接展示给用户；随后，模型基于这段推理生成最终的文本回复（即 `TextBlock`），这才是用户真正看到的内容。

```
1 response_content = history[-1].content
2 for block in response_content:
3     if hasattr(block, "text"):
4         print(block.text)
```

Listing 4: 响应体遍历与文本提取逻辑示意

上述代码片段展示了程序如何处理 `response_content`。它通过一个 `for` 循环遍历列表中的每一个 `block`，并检查该 `block` 是否具有 `text` 属性。如果存在，则将其打印输出。²⁴

4.3 第一个块：ThinkingBlock（思考块）

在遍历的第一个元素中，`block` 的类型为 `ThinkingBlock`。这是模型的思考块，其中记录了大语言模型在生成最终答案之前的内部推理过程。通过调试面板展开该对象可以发现，`ThinkingBlock` 中并没有 `text` 这个属性字段。因此，当程序执行到 `if hasattr(block, "text")` 这一判断时，条件不成立，该块不会被打印出来。²⁵

`ThinkingBlock` 的存在体现了现代大模型的一种设计模式：将“思考”与“回答”分离。思考过程可能包含对用户意图的分析、对上下文的理解、对回答策略的权衡等，这些内容对于模型的生成质量至关重要，但通常不适合直接暴露给终端用户。因此，程序在解析响应时需要能够识别并跳过这类块。

4.4 第二个块：TextBlock（文本块）

继续遍历到第二个元素时，`block` 的类型变为 `TextBlock`，即文本块。与 `ThinkingBlock` 不同，`TextBlock` 明确包含一个 `text` 属性。在调试面板中展开该对象，可以看到 `text` 的值为：

```
"Hello, how can I help you today?"
```

²³视频时间戳：04:41

²⁴视频时间戳：05:01

²⁵视频时间戳：05:11

这正是模型对我们输入的 "Hello" 给出的最终回复。当程序检测到该块具有 `text` 属性后，便执行打印操作，将上述问候语输出到控制台。²⁶

至此，`response_content` 列表中的两个元素均已遍历完毕。由于列表长度仅为 2，循环在第二次迭代后自然结束，程序继续向下执行，完成本轮对话的输出流程。

4.5 本章小结

本章通过逐行 Debug，揭示了模型响应在 `history` 中的存储形态，以及程序如何通过遍历 `response_content` 来分离 `ThinkingBlock` 与 `TextBlock`。核心要点可以总结为以下三点：

1. `agent_loop` 返回后，`history[-1]` 即为模型响应，其 `content` 是一个 `Block` 列表；
2. `ThinkingBlock` 包含模型的内部思考过程，没有 `text` 属性，不直接展示给用户；
3. `TextBlock` 包含 `text` 属性，存储模型最终要输出的回答文本，如 "Hello, how can I help you today?"。

这一解析机制是 Agent 与 LLM 交互中的关键环节。它不仅决定了用户最终看到什么，也为后续处理更复杂的响应结构（例如包含工具调用信息的块）奠定了基础。在下一章中，我们将基于前四章的内容，对整个无 **Tool Use** 场景的交互流程进行系统总结，并预告 `tool_use` 分支的核心地位。

5 流程总结与下节预告

本节课程通过 Debug 的方式，完整追踪了用户输入 "Hello" 后，程序在不涉及 `tool_use` 场景下的执行路径。UP 主在视频末尾对前四章的 Debug 过程进行了系统性复盘²⁷，并自然引出了下一节的核心内容。本章将对这一完整流程进行梳理与总结。

5.1 无 Tool Use 场景的完整交互流程

当用户在命令行中输入 "Hello" 并回车后，程序依次经历了以下关键步骤：

1. **外层循环接收输入。**主程序中的 `while True` 循环（根据视频画面中的第 92 行）持续运行，等待用户输入。输入内容被存储在 `query` 变量中。
2. **封装消息字典。**程序将用户的输入封装为一条消息字典 `{"role": "user", "content": query}`，并追加到 `history` 列表中（第 99 行）。此时 `history` 中包含第一条消息。
3. **进入 `agent_loop`。**主程序在第 100 行调用 `agent_loop(history)`，将历史消息传入 **智能体循环**（Agent Loop）。这是整个 Agent 运行的核心逻辑。
4. **发送请求并接收响应。**`agent_loop` 内部首先通过 **客户端**（Client）向配置的 GLM 后端发送请求，获取大模型返回的 `response` 对象。

²⁶视频时间戳：05:25

²⁷视频时间戳：05:00

5. 追加响应到消息列表。在 `agent_loop` 内部第 74 行，程序将 `response` 追加到 `messages` 列表（即外部传入的 `history` 的引用）。此时消息列表中已有两条记录。
6. 判断停止原因。程序检查 `response.stop_reason` 的值。由于输入 "Hello" 无需调用任何工具，模型返回的 停止原因（Stop Reason）为 "end_turn"（结束回合），表示本轮对话正常结束²⁸。
7. 退出内层循环。因为 `stop_reason` 不是 "tool_use"，程序执行第 77 行的 `return`，跳出 `agent_loop`，回到主程序。
8. 解析响应内容体。主程序通过 `history[-1].content` 获取模型响应的内容体，即 `response_content`。它是一个 `Block` 列表，包含两个元素：`ThinkingBlock` 和 `TextBlock`。
9. 分离思考与回答并输出。程序遍历 `response_content`，检查每个 `block` 是否具有 `text` 属性。第一个 `ThinkingBlock`（思考块）仅包含模型的内部思考过程，没有 `text` 字段；第二个 `TextBlock`（文本块）包含 `text` 属性，其值为 "Hello, how can I help you today?"。程序将其打印输出²⁹。

以上九个步骤，构成了一个完整的、无 `tool_use` 的用户查询（Query）交互闭环。从 `query` 的获取、`history` 的构建、`agent_loop` 的请求发送，到 `response` 的接收、`stop_reason` 的判断，再到 `ThinkingBlock` 与 `TextBlock` 的解析与输出，每一个环节都环环相扣。

5.2 下节预告：Tool Use 分支

UP 主在总结时特别指出，虽然本节详细讲解了 `agent_loop` 中 `stop_reason == "end_turn"` 的分支，但 `agent_loop` 中真正更核心的部分，其实是处理 "tool_use" 的分支³⁰。当模型判断需要调用外部工具时，`stop_reason` 会被设为 "tool_use"，Agent 将进入工具调用分支执行相应操作——这才是 Agent 能够完成复杂任务的关键所在。下一节课程将以此为重点，深入 Debug 工具使用（Tool Use）的源码逻辑。

²⁸视频时间戳：03:42

²⁹视频时间戳：05:25

³⁰视频时间戳：06:35